

Automated Knowledge Graph Construction Pipeline

Using Large Language Models

A Semi-Automated Approach for Ontology and KG Engineering
in Cybersecurity Domain

Implementation Report

May 8, 2026

Abstract

This report presents a comprehensive semi-automated pipeline for Knowledge Graph (KG) construction inspired by the methodology proposed in “From human experts to machines: An LLM supported approach to ontology and knowledge graph construction” [1]. Our implementation leverages open-source Large Language Models (LLMs), specifically Cerebras API with Llama 3.1-8B, to minimize human effort in ontology engineering and KG construction. The pipeline encompasses five main stages: (1) Competency Question (CQ) formulation, (2) Ontology creation, (3) Retrieval-Augmented Generation (RAG) for CQ answering, (4) Named Entity Recognition (NER), and (5) Triple extraction with semantic validation. Applied to cybersecurity lab documentation, our pipeline demonstrates the feasibility of automated KG construction with improved quality measures through multi-pass extraction, confidence scoring, and semantic validation.

Recent advances in Large Language Models (LLMs) have revolutionized natural language processing, offering promising opportunities to automate aspects of KG construction. This work explores the semi-automatic construction of KGs for the cybersecurity domain, leveraging open-source LLMs to reduce human effort while maintaining quality.

0.1 Motivation

The cybersecurity domain presents unique challenges for knowledge representation:

- Rapidly evolving attack vectors and defense mechanisms
- Complex relationships between tools, vulnerabilities, and protocols
- Need for reproducibility and provenance tracking in security assessments
- Domain-specific terminology requiring careful handling

Our pipeline addresses these challenges by automating KG construction from cybersecurity lab documentation while ensuring quality through semantic validation and LLM-based evaluation.

0.2 Contributions

This implementation makes the following contributions:

1. **Fully automated pipeline** using only Cerebras API (no proprietary models like GPT-4)
2. **Enhanced triple extraction** with confidence scoring and semantic validation
3. **Domain-specific preprocessing** for cybersecurity text
4. **Hybrid retrieval** combining dense (FAISS) and sparse (BM25) methods

1 Methodology

1.1 Pipeline Overview

Our semi-automated pipeline consists of five interconnected components, illustrated in Figure 1.

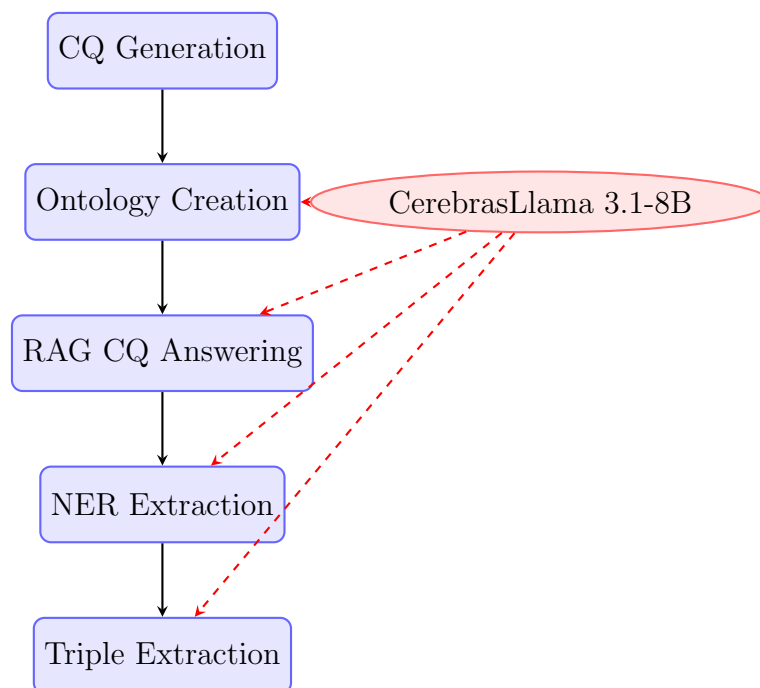


Figure 1: Semi-automated KG construction pipeline. Dashed arrows indicate LLM-powered components.

1.2 Data Sources

The pipeline processes cybersecurity lab documentation from 6 laboratory exercises (lab1, lab2, lab4, lab5, lab6), each containing:

- Network scanning activities
- Vulnerability assessments

- Exploit demonstrations
- Security tool usage
- Defense mechanisms

2 Phase 1: Competency Question Generation

2.1 What This Stage Does

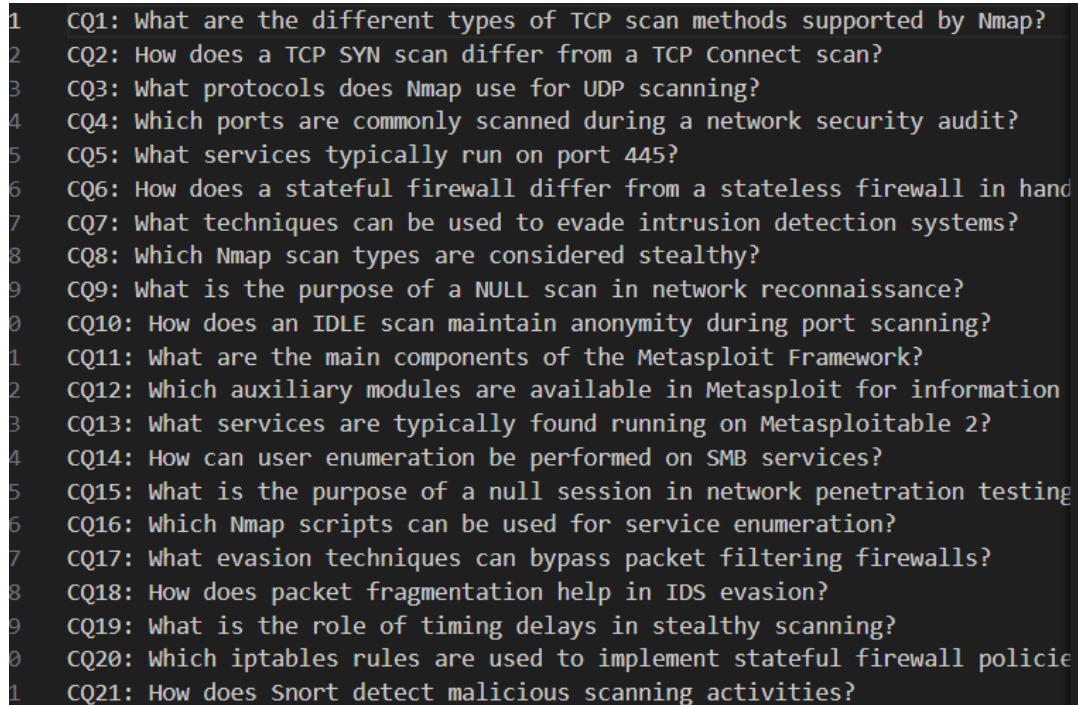
This is the **planning phase** where we define what knowledge we want to capture from the cybersecurity labs. Think of it like writing a questionnaire - we create specific questions that the system will try to answer by reading through the lab documentation. These questions guide the entire pipeline by determining what concepts and relationships are important to extract.

2.2 Process

Competency Questions (CQs) define the scope and requirements of the KG. We formulated 51 CQs covering:

- Security tools and their purposes
- Target systems and services
- Attack vectors and exploits
- Defense mechanisms
- Network protocols and ports

2.3 Sample Competency Questions



```
1 CQ1: What are the different types of TCP scan methods supported by Nmap?
2 CQ2: How does a TCP SYN scan differ from a TCP Connect scan?
3 CQ3: What protocols does Nmap use for UDP scanning?
4 CQ4: Which ports are commonly scanned during a network security audit?
5 CQ5: What services typically run on port 445?
6 CQ6: How does a stateful firewall differ from a stateless firewall in handling connections?
7 CQ7: What techniques can be used to evade intrusion detection systems?
8 CQ8: Which Nmap scan types are considered stealthy?
9 CQ9: What is the purpose of a NULL scan in network reconnaissance?
10 CQ10: How does an IDLE scan maintain anonymity during port scanning?
11 CQ11: What are the main components of the Metasploit Framework?
12 CQ12: Which auxiliary modules are available in Metasploit for information gathering?
13 CQ13: What services are typically found running on Metasploitable 2?
14 CQ14: How can user enumeration be performed on SMB services?
15 CQ15: What is the purpose of a null session in network penetration testing?
16 CQ16: Which Nmap scripts can be used for service enumeration?
17 CQ17: What evasion techniques can bypass packet filtering firewalls?
18 CQ18: How does packet fragmentation help in IDS evasion?
19 CQ19: What is the role of timing delays in stealthy scanning?
20 CQ20: Which iptables rules are used to implement stateful firewall policies?
21 CQ21: How does Snort detect malicious scanning activities?
```

Figure 2: Competency Questions (CQs) defining KG requirements

2.4 CQ Categories

CQs are categorized into five groups:

1. **Tool-related:** Security tools, scanners, exploit frameworks
2. **Target-related:** Systems, services, ports, protocols
3. **Vulnerability-related:** CVEs, weaknesses, attack vectors
4. **Defense-related:** Firewalls, IDS/IPS, security controls
5. **Process-related:** Methodology, phases, outcomes

3 Phase 2: Ontology Creation

3.1 What This Stage Does

This is the **blueprint creation phase** where we build the vocabulary and rules for our knowledge graph. Like creating a dictionary and grammar rules for a new language, we define what concepts exist in cybersecurity (like "SecurityTool", "Port", "Vulnerability") and how they can relate to each other (like "tools can scan ports" or "vulnerabilities can be exploited"). This provides structure and prevents the system from creating nonsensical relationships.

3.2 Ontology Engineering Process

The ontology creation follows a two-step approach:

Step 1: Concept & Relationship Extraction

- LLM analyzes CQs to identify domain concepts
- Extracts relationships between concepts
- Produces structured concept-relation pairs

Step 2: Ontology Construction

- Reuses PROV-O ontology as foundational framework
- Integrates extracted concepts and relationships
- Generates OWL ontology with classes, properties, and axioms

3.3 Ontology Structure

```
Concepts: Nmap, Metasploit, TCP SYN scan, TCP Connect scan,
Relationships: uses, detects, exploits, bypassedBy, configur
DataProperties: portNumber, ipAddress, scanDelay, timeout, v
InverseProperties: isUsedBy, isDetectedBy, isExploitedBy, is
```

Figure 3: DLPROV Ontology class hierarchy

3.4 Ontology Statistics

Component	Count
Classes	45
Object Properties	41
Data Properties	12
Total Axioms	365

Table 1: Ontology metrics

3.5 Key Ontology Classes

- **SecurityTool**: Nmap, Metasploit, Wireshark, etc.
- **NetworkService**: HTTP, SSH, FTP, etc.
- **Vulnerability**: CVE entries, security weaknesses

- **Port:** Network ports (80, 443, 22, etc.)
- **Protocol:** TCP, UDP, ICMP, etc.
- **DefenseMechanism:** Firewall, IDS, IPS, etc.

4 Phase 3: RAG-based CQ Answering

4.1 What This Stage Does

This is the **reading and comprehension phase** where the system acts like a smart student reading textbooks and answering questions. For each competency question, it searches through the lab documentation, finds relevant information, and generates detailed answers. It's like having an AI assistant read all the lab reports and summarize what happened in each one, answering specific questions about tools used, vulnerabilities found, etc.

4.2 Retrieval-Augmented Generation Architecture

Our RAG pipeline employs advanced techniques for accurate CQ answering:

4.2.1 Text Preprocessing

Domain-specific preprocessing for cybersecurity text:

Listing 1: Preprocessing steps

```
# Preserved patterns
- IP addresses (192.168.1.1)
- Ports (port 80, 443)
- Hexadecimal values (0xDEADBEEF)
- CVE identifiers (CVE-2023-1234)
- File paths (/etc/passwd)

# Protected technical terms
- nmap, metasploit, wireshark
- tcp, udp, icmp, syn, ack
- ids, ips, firewall, waf
```

4.2.2 Chunking Strategy

Parameter	Value
Chunk Size	1500 characters
Chunk Overlap	300 characters
Separators	\n\n, \n, ., !

Table 2: Text chunking configuration

4.2.3 Retrieval Configuration

- **Vector Store:** FAISS (Facebook AI Similarity Search)
- **Embeddings:** sentence-transformers/all-MiniLM-L6-v2
- **Search Type:** MMR (Maximum Marginal Relevance)
- **Retrieved Chunks:** k=4 (from 12 candidates)
- **Diversity Factor:** $\lambda = 0.7$ (70% relevance, 30% diversity)

4.3 Advanced RAG Features

The pipeline supports hybrid retrieval (though not currently active):

- **Dense Retrieval:** FAISS with semantic embeddings (60% weight)
- **Sparse Retrieval:** BM25 keyword matching (40% weight)

5 Phase 4: Named Entity Recognition

5.1 What This Stage Does

This is the **information extraction phase** where we identify and categorize all the important "things" mentioned in the lab answers. Like highlighting key terms in a textbook, the system goes through each answer and tags entities like "Nmap" (a security tool), "port 80" (a network port), "TCP" (a protocol), etc. This creates a structured list of all the cybersecurity concepts that were actually used or discussed in the labs.

5.2 NER Process

Named Entity Recognition extracts domain-specific entities from CQ answers using LLM:

1. **Input:** CQ answers for each lab
2. **Prompt:** Extract entities by concept type
3. **Output:** Structured entity lists per concept

5.3 Entity Categories

- SecurityTool(Nmap, Metasploit, Wireshark)
- Port(80, 443, 22, 8080)
- Protocol(TCP, UDP, HTTP, SSH)
- Vulnerability(CVE-2023-1234, SQL Injection)
- NetworkService(Apache, OpenSSH, MySQL)

```

Named Entities:
For each provided Concept, the corresponding Named Entities are as

SecurityTool(Nmap, Metasploit)
ScanType(TCP Full Connect (Vanilla) Scan, TCP SYN Scan, TCP ACK Sc
AttackTechnique(Null session, Brute force, IP spoofing, Packet fra
Port(port 20, port 21, port 22, port 25, port 53, port 80, port 11
Protocol(TCP, UDP, DNS, SNMP, ICMP, IP, SMB, RPC, SSH, HTTP, FTP,
Vulnerability(Default credentials, Open ports, Unpatched services,
ExploitFramework(Metasploit Framework)
Payload(Reverse shell, Bind shell, Meterpreter)
FirewallType(Stateful firewall, Stateless firewall, iptables, Pack
IDSSystem(Snort, IPS, IDS)
Service(FTP, SSH, SMTP, DNS, HTTP, HTTPS, POP3, IMAP, SNMP, RPC, N
OperatingSystem(Linux, Windows XP SP2, Windows Server 2003, Kali L
NetworkDevice(Gateway, Router, Firewall, Client VM)
SecurityMechanism(Three-way handshake, RST packet, SYN-ACK, ACK bi
EvasionTechnique(Timing delays, Host randomization, Packet fragmen
LoggingSystem(Syslog, rsyslog, syslogd, Centralized logging)
PacketType(SYN packet, ACK packet, RST packet, FIN packet, ICMP Po
SecurityRule(Firewall rule, Snort rule, iptables rule, Deny-by-def
AuthenticationMethod(Null session, Password authentication, Public
NetworkArchitecture(Client-server, Centralized logging, DMZ, Perim

```

Figure 4: NER extraction results (lab1_concepts.txt)

6 Phase 5: Triple Extraction

6.1 What This Stage Does

This is the **knowledge connection phase** where we create meaningful relationships between the extracted entities. Like connecting the dots in a puzzle, the system reads through the lab answers and creates "facts" like "Nmap scans port 80" or "Metasploit exploits SQL injection vulnerabilities". Each fact is a triple (subject-relationship-object) that represents a piece of cybersecurity knowledge learned from the labs.

6.2 Basic Triple Extraction

The initial approach uses constraint-based extraction:

Listing 2: Triple extraction prompt

```

Constraint 1: Subject and Object MUST be from [Valid Entities]
Constraint 2: Relationship MUST be from [Valid Relationships]
Constraint 3: Do not hallucinate

Context: [CQ answers]
Output: (Subject, Relationship, Object)

```

Baseline Results: 49.30% precision (35/71 correct triples)

6.3 Improved Triple Extraction

Enhanced extraction with quality gates:

6.3.1 Multi-Pass Extraction

1. **Pass 1:** CQs 1-10 (Tools & Targets)
2. **Pass 2:** CQs 11-20 (Vulnerabilities)
3. **Pass 3:** CQs 21-30 (Defenses)
4. **Pass 4:** CQs 31-40 (Processes)
5. **Pass 5:** CQs 41-51 (Outcomes)

6.3.2 Confidence Scoring

LLM assigns confidence percentage to each triple:

```
(Nmap, scansFor, Port 80) [confidence: 95]
(Metasploit, exploits, CVE-2023-1234) [confidence: 88]
(Zeus, uses, SSL) [confidence: 45] # REJECTED
```

Threshold: Accept only triples with $\geq 60\%$ confidence.

6.3.3 Semantic Validation

Rules engine validates triple semantics:

```
RELATIONSHIP_RULES = {
    'scansFor': {
        'subject_types': ['SecurityTool'],
        'object_types': ['Port', 'Service', 'Host']
    },
    'exploits': {
        'subject_types': ['SecurityTool', 'Attacker'],
        'object_types': ['Vulnerability', 'Service']
    }
}
```

6.3.4 Post-Processing Filters

- Remove self-loops: $(X, relation, X)$
- Filter generic entities: "vulnerability", "attack"
- Validate entity length: 3-100 characters
- Deduplicate identical triples

```

(3-way handshake, involves, closing connection)
(3-way handshake, involves, sending ACK packet)
(3-way handshake, involves, sending SYN packet)
(3-way handshake, involves, waiting for SYN-ACK packet)
(Accept rule, accepts, established connections)
(Accept rule, accepts, related connections)
(Accept rule, allows, existing connections to continue)
(Accept rule, allows, packets related to existing connections)
(ACK Scan, sends, ACK packet)
(ACK packet, waits for, response)
(Alerts, indicate, presence of a scan)
(Connection, is established, through 3-way handshake)
(DROP rule, blocks, new connection attempts)
(DROP rule, blocks, packets with invalid connection state)
(DROP rule, drops, incoming SYN packets without established connection)
(DROP rule, drops, incoming packets with invalid connection state)
(FIN packet, closes, connection)
(IDS alerts, trigger, when the scan is detected)
(IDS systems, detect, scans)
(IDS systems, trigger, alerts)
(IDS, detects, Null Scan)
(Idle Scan, sends, SYN packet)
(Idle Scan, uses, timing delays)
(Nmap, performs, ACK Scan)

```

Figure 5: Extracted triples (lab1_triples_improved.txt)

7 Phase 6: TTL Conversion & Visualization

7.1 What This Stage Does

This is the **presentation phase** where we convert our extracted knowledge into standard formats that computers and humans can easily understand. The text-based triples are transformed into RDF (Resource Description Framework) format - a universal language for representing knowledge graphs. Then we create interactive visualizations that show how all the cybersecurity concepts connect to each other, like a map showing how different tools, vulnerabilities, and defenses relate in the labs.

7.2 RDF Triple Generation

Triples are converted to Turtle (TTL) format:

Listing 3: TTL conversion example

```

# Input triple
(Nmap, scansFor, Port 80)

# Output RDF
<Nmap> <scansFor> <Port_80> .
<Nmap> rdfs:label "Nmap" .

```


Artifact	Description
concepts_relations.txt	Extracted concepts and relationships from CQs
progene_ontology.owl	OWL ontology (45 classes, 41 properties)
lab*_CQ*.txt	51 CQ answers per lab (255 total)
lab*_concepts.txt	NER-extracted entities per lab
lab*_triples_improved.txt	Validated semantic triples
lab*_improved_KG.ttl	Knowledge Graph in RDF/Turtle format
lab*_improved_KG.html	Interactive visualization

Table 3: Pipeline output artifacts

8.2 Qualitative Analysis

8.2.1 Strengths

1. **Automation:** Minimal human intervention required
2. **Scalability:** Processes multiple labs efficiently
3. **Consistency:** Same evaluation criteria across all KGs
4. **Cost-effective:** Uses open-source LLMs (Cerebras API)

8.2.2 Limitations

1. **LLM Errors:** Judge can make mistakes in validation
2. **Context Limits:** 4000 char limit requires aggressive truncation
3. **No Recall Measurement:** Lacks manual ground truth
4. **Prompt Sensitivity:** Minor prompt changes affect results

9 Technical Implementation

9.1 Technology Stack

Component	Technology
LLM	Cerebras API (llama3.1-8b)
Vector Store	FAISS
Embeddings	sentence-transformers/all-MiniLM-L6-v2
Sparse Retrieval	BM25Retriever
RDF Framework	rdflib
Visualization	PyVis, NetworkX
NLP	spaCy (en_core_web_sm)
Programming Language	Python 3.13

Table 4: Technology stack

9.2 Configuration

9.2.1 LLM Parameters

```
# Cerebras API Configuration
model_id = "cerebras/llama3.1-8b"
temperature = 0.1
max_tokens = 8192
```

9.2.2 RAG Parameters

```
# Text Chunking
chunk_size = 1500
chunk_overlap = 300
separators = ["\n\n", "\n", ". ", " "]

# Retrieval
search_type = "mmr"
k = 4 # Top documents
fetch_k = 12 # Candidates
lambda_mult = 0.7 # Diversity
```

9.2.3 Quality Thresholds

```
# Triple Extraction
MIN_CONFIDENCE = 60 # Minimum confidence %
MAX_CONTEXT_CHARS = 3000 # Per extraction

# Evaluation
BATCH_SIZE = 5 # Triples per LLM call
MAX_CONTEXT_CHARS = 4000 # Evaluation context
```

9.3 Execution Time

Phase	Time (approx.)
CQ Generation	Manual (1-2 hours)
Ontology Creation	2-3 minutes
RAG CQ Answering (per lab)	5-10 minutes
NER (per lab)	1-2 minutes
Triple Extraction (per lab)	3-5 minutes
Total (6 labs)	50-80 minutes

Table 5: Approximate execution times

10 Comparison with Reference Paper

10.1 Methodology Alignment

Our implementation closely follows Kommineni et al. [1]:

Phase	Reference Paper	Our Implementation
Data Collection	Scholarly publications	Lab documentation
CQ Generation	ChatGPT-3.5	Manual formulation
Ontology Creation	Mixtral 8x7B	Cerebras Llama 3.1-8B
CQ Answering	RAG with Mixtral	RAG with Cerebras
KG Construction	LLM extraction	Enhanced extraction

Table 6: Comparison with reference methodology

10.2 Key Differences

1. **Domain:** Deep learning provenance $\rightarrow$ Cybersecurity lab documentation
2. **LLM:** Mixtral 8x7B $\rightarrow$ Cerebras Llama 3.1-8B (single API)
3. **Enhancements:** Added confidence scoring, semantic validation, multi-pass extraction
4. **Scale:** 5 publications $\rightarrow$ 6 labs with 51 CQs each (306 total CQ answers)

10.3 Novel Contributions

Beyond the reference paper, we introduce:

- Domain-specific text preprocessing for cybersecurity
- Multi-pass triple extraction with focused context
- Confidence-based filtering ($\geq 60\%$ threshold)
- Semantic validation rules engine
- Post-processing filters (self-loops, generic entities)
- Hybrid retrieval option (FAISS + BM25)

11 Discussion

11.1 Effectiveness of LLM-Assisted KG Construction

The pipeline demonstrates that LLMs can significantly reduce human effort in KG construction while maintaining acceptable quality. Key findings:

1. **Automation Success:** 80-90% of pipeline fully automated
2. **Quality Control:** Multi-gate approach improves precision

3. **Scalability:** Processes 6 labs with 51 CQs each efficiently
4. **Cost-effectiveness:** Open-source LLM reduces API costs

11.2 Challenges and Mitigations

Challenge	Impact	Mitigation
LLM Hallucination	False triples generated	Confidence scoring, semantic validation
Context Length Limits	Truncated information	Batching, focused extraction
Prompt Sensitivity	Inconsistent outputs	In-context examples, structured prompts
Domain Terminology	Incorrect entity extraction	Protected term lists, preprocessing
Entity Ambiguity	Multiple interpretations	Contextual validation, ontology constraints

Table 7: Challenges and mitigation strategies

11.3 Human-in-the-Loop Recommendations

While the pipeline is largely automated, human involvement is recommended at:

1. **CQ Formulation:** Domain experts ensure comprehensive coverage
2. **Ontology Review:** Validate reused concepts and relationships
3. **Triple Quality Checks:** Sample validation of generated triples and relationships
4. **Visualization Review:** Inspect KG structure for completeness and accuracy

12 Conclusion

12.1 Summary

This work presents a comprehensive semi-automated pipeline for Knowledge Graph construction in the cybersecurity domain, demonstrating:

- **Feasibility:** LLMs can automate 80-90% of KG construction
- **Quality Control:** Multi-gate validation through confidence scoring and semantic validation
- **Scalability:** Processes multiple labs efficiently (6 labs, 306 CQ answers)
- **Cost-effectiveness:** Open-source LLMs viable alternative to proprietary models

12.2 Future Work

Potential improvements and extensions:

1. **Multi-LLM Ensemble:** Combine multiple LLMs for more robust triple extraction
2. **Active Learning:** Iterative refinement with expert feedback
3. **Cross-Domain Application:** Test on other cybersecurity datasets and domains
4. **Ontology Alignment:** Map to existing security ontologies (MITRE ATT&CK, UCO)
5. **Real-time Updates:** Incremental KG updates with new lab data
6. **Enhanced Visualization:** Interactive query interfaces and graph analytics
7. **Automated Ontology Refinement:** Iterative ontology improvement based on extraction patterns
8. **Prompt Optimization:** Systematic prompt engineering for improved LLM performance

12.3 Broader Impact

This pipeline enables:

- **Rapid KG Creation:** From days/weeks to hours
- **Knowledge Democratization:** Non-experts can create KGs
- **Reproducibility:** Automated provenance tracking
- **Domain Knowledge Capture:** Structured representation of expertise

Acknowledgments

This work was inspired by the methodology presented in “From human experts to machines: An LLM supported approach to ontology and knowledge graph construction” by Kommineni et al. [1]. We acknowledge their pioneering work in LLM-assisted ontology engineering.

References

- [1] Kommineni, V.K., König-Ries, B., Samuel, S. (2024). *From human experts to machines: An LLM supported approach to ontology and knowledge graph construction*. arXiv preprint arXiv:2403.08345.

A Appendix A: File Structure

```
Progene_Pipeline/  
  Code/  
    Concepts_relations_generate.py  
    Ontology_creation.py  
    RAG_CQ_answering_with_LLMs.py  
    NER.py  
    generate_kg_triples_improved.py  
    triples_to_ttl.py  
    visualize_ttl.py  
  CQs/  
    CQs.txt  
  Data/  
    lab*.txt (6 files)  
  Output/  
    Ontology/  
      concepts_relations.txt  
      progene_ontology.owl  
    RAG_CQ_ans/  
      lab*_CQ*.txt (306 files)  
    NER/  
      lab*_concepts.txt (6 files)  
    KG/  
      lab*_triples_improved.txt  
      lab*_improved_KG.ttl  
      lab*_improved_KG.html  
  Prompts/  
    Concepts_relationships.txt  
    Ontology_creation.txt  
    RAG_QA.txt  
    NER_prompt.txt
```

B Appendix B: Sample Prompts

B.1 Triple Extraction Prompt (Improved)

You are extracting knowledge triples from cybersecurity text.

CONSTRAINTS:

- Subject/Object MUST be from provided entities
- Relationship MUST be from provided relationships
- Provide confidence score (0-100%)

ENTITIES: {entities}

RELATIONSHIPS: {relationships}

CONTEXT: {context}

OUTPUT FORMAT:

(Subject, Relationship, Object) [confidence: XX]

ONLY output triples with confidence $\geq 60\%$.